

Java Full Stack Development

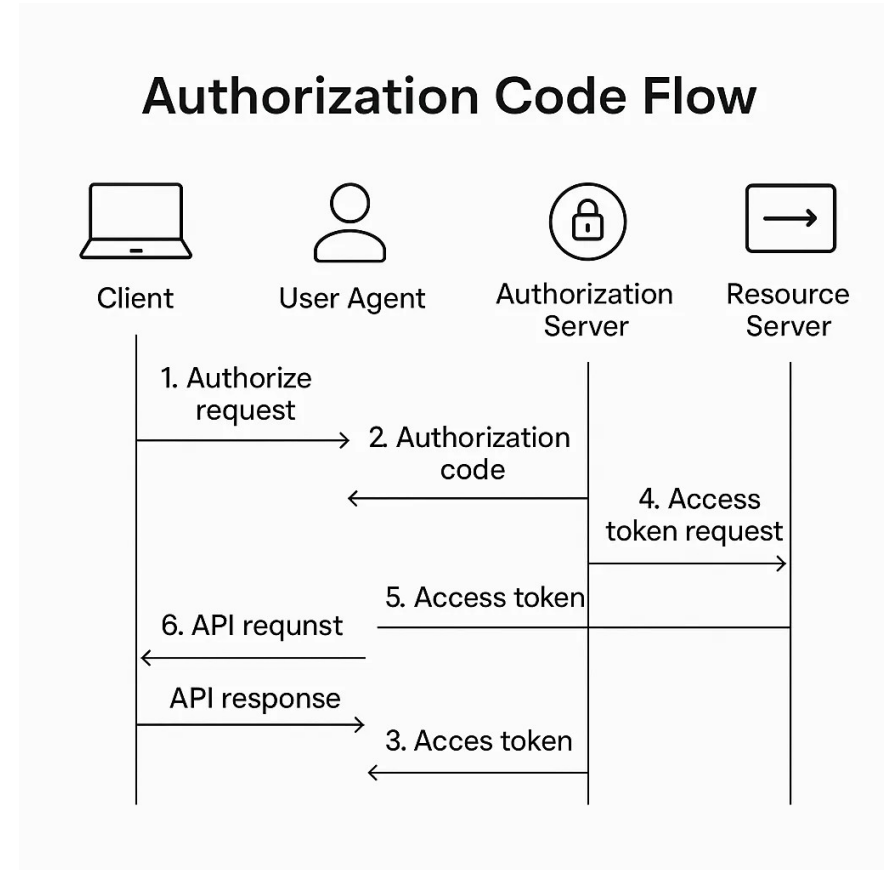
2. Oauth Hardening





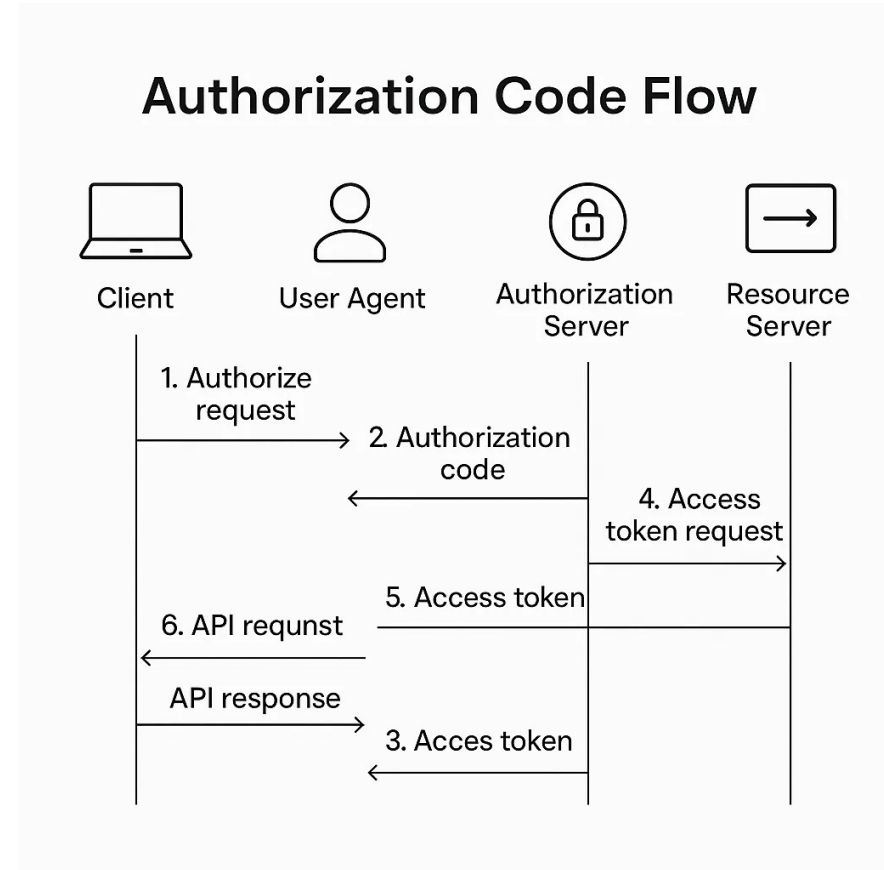
Our Stack

- Our labs
 - Auth server
 - In-memory users
 - Hardcoded client secrets
 - RSA key generated fresh each startup
 - Resource server
 - Validates JWTs against the auth server's JWKS
 - No audience validation



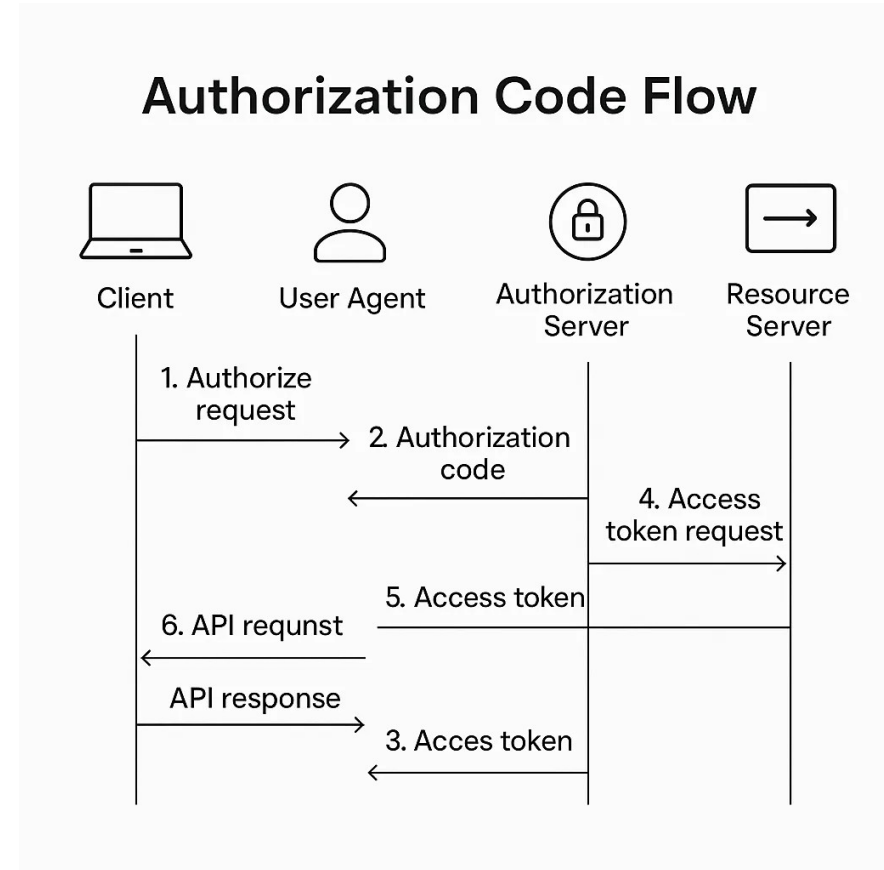
Our Stack

- Our labs
 - BFF
 - Stores tokens in HTTP session, default cookie attributes, default token timeouts
 - All three
 - Running on localhost
 - No HTTPS
 - No rate limits
 - No audit logging



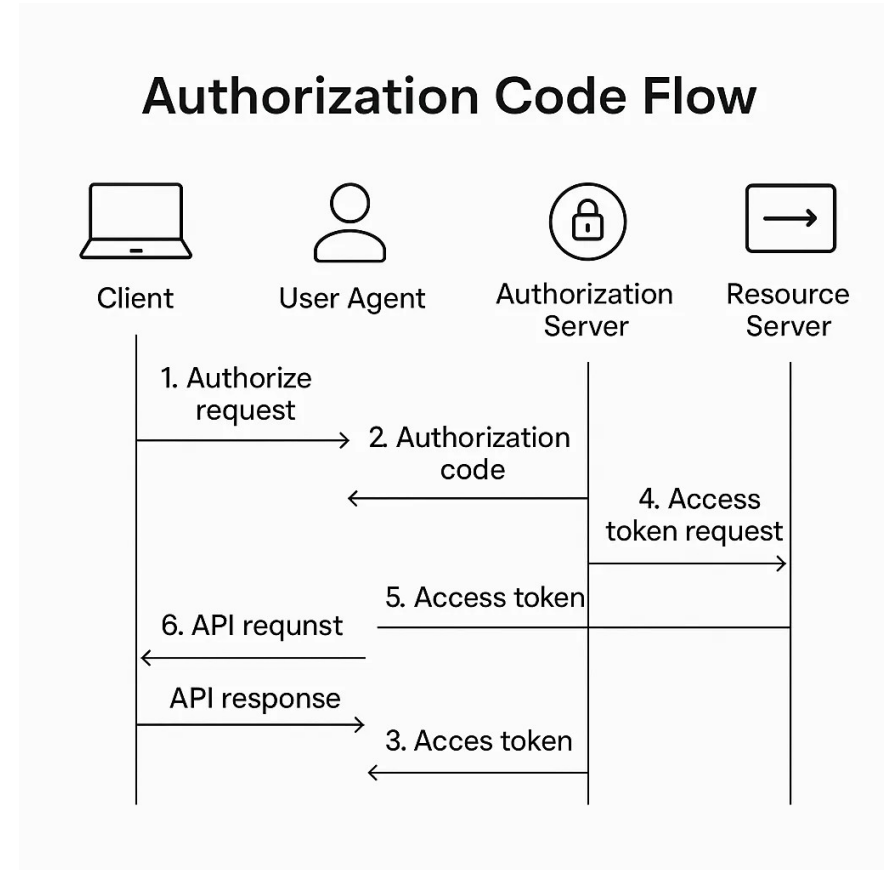
Our Stack

- Auth server compromises
 - The auth server uses in-memory users
 - Alice, bob with hardcoded passwords
 - It uses {noop} password encoding which Spring explicitly warns against in production
 - It generates a fresh RSA key pair on every startup, which means every restart invalidates all previously-issued tokens
 - None of these are appropriate for production
 - But these options kept the lab simpler.



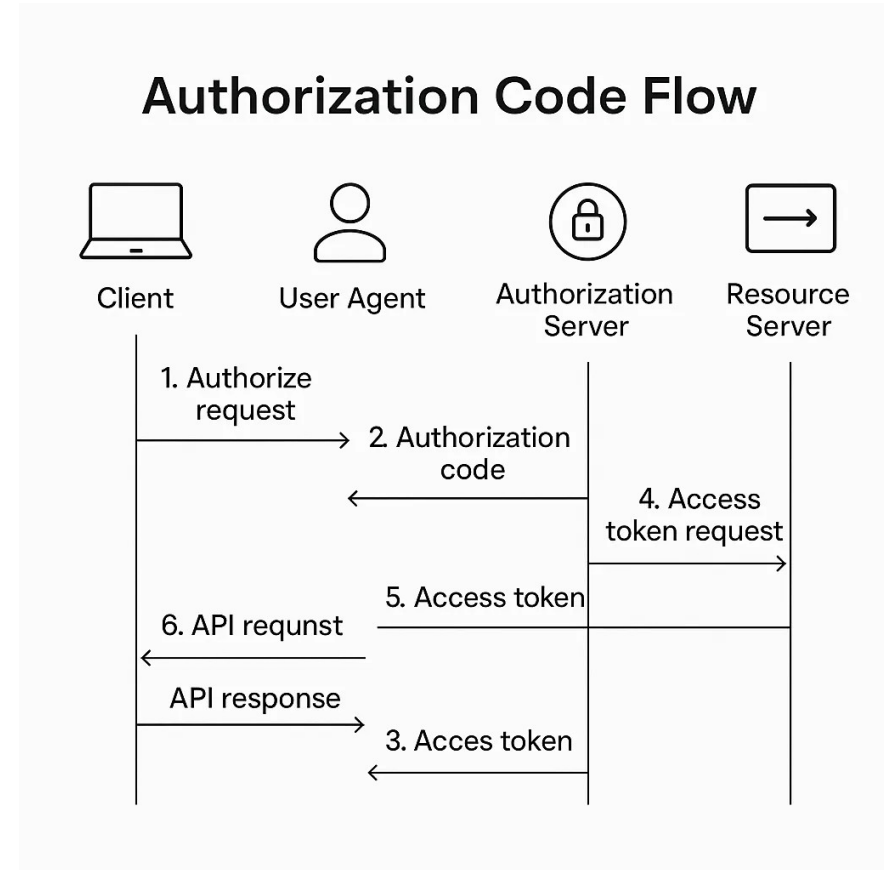
Our Stack

- Resource server compromises
 - The bankserver validates JWT signatures against the auth server's JWKS endpoint and checks that the issuer claim matches
 - It does not validate the audience claim
 - This means a token issued for the bankbff client could theoretically be replayed against a different resource server that trusts the same auth server
 - A security gap that audience validation would close.



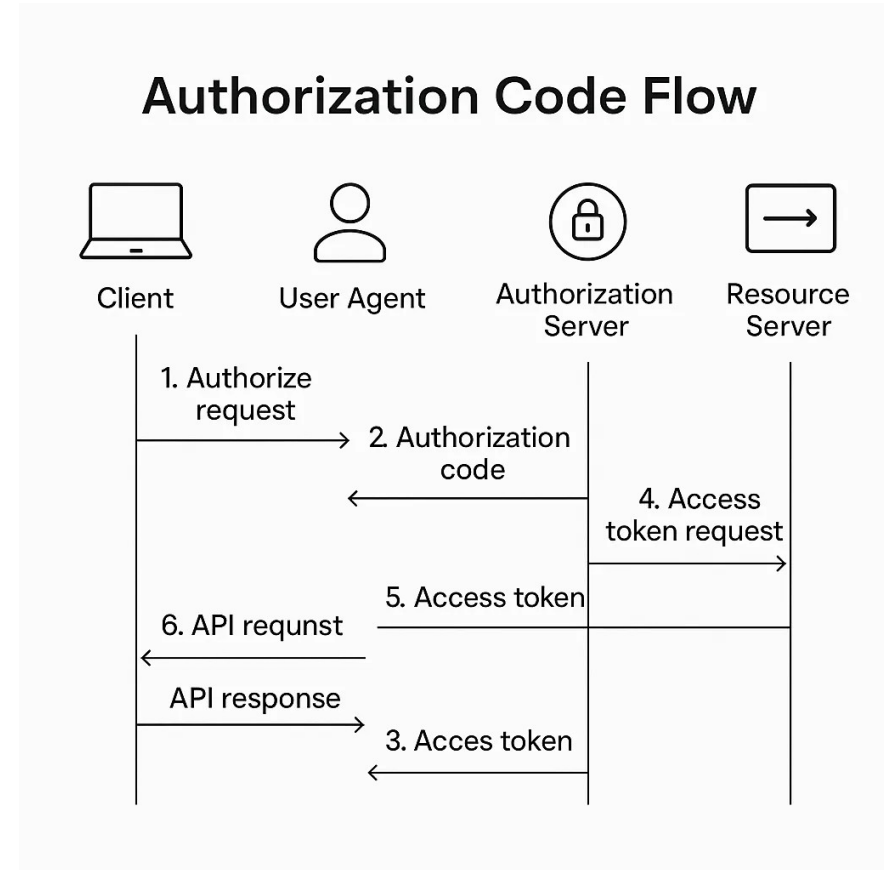
Our Stack

- BFF compromises
 - The bankbff stores OAuth tokens in the HTTP session
 - Which is the default Spring Security behavior.
 - The session cookie is HttpOnly (correct)
 - But uses default settings for everything else
 - No Secure flag which is acceptable on localhost, wrong for production
 - SameSite=Lax by default
 - Sometimes correct, sometimes wrong, but always worth being explicit
 - Token lifetimes use the auth server's default of 60 minutes for access tokens, 24 hours for refresh tokens



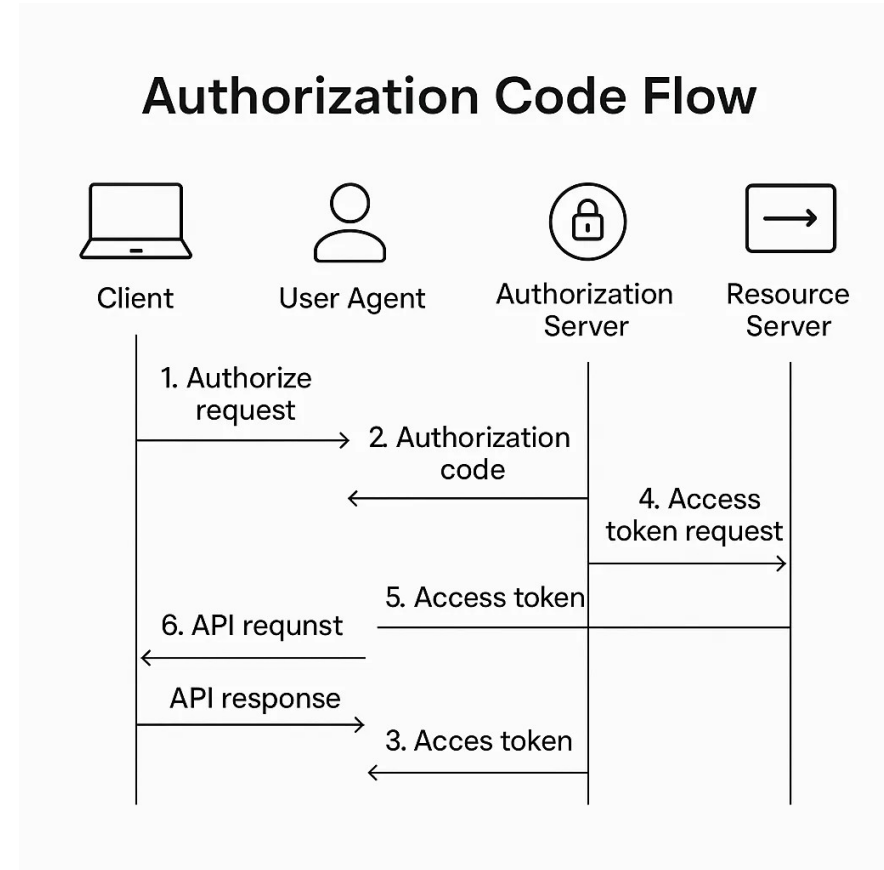
Our Stack

- Cross-cutting compromises
 - Everything runs on <http://localhost>
 - There is no HTTPS, so all traffic including cookies and tokens is in plaintext.
 - There are no rate limits
 - So an attacker could brute-force credentials or pound the token endpoint
 - There is no audit logging of security-relevant events



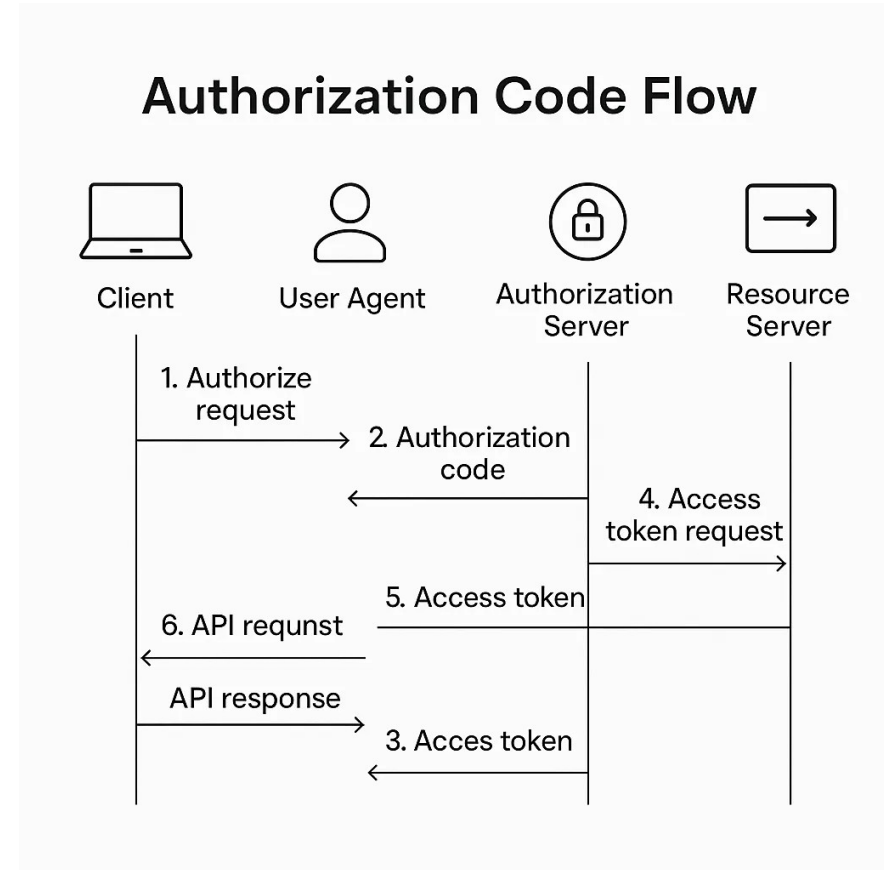
Token Lifetime Tuning

- How long should tokens live?
 - Access tokens
 - Short, minutes, not hours
 - 5-15 minutes is typical for high-security applications
 - Refresh tokens
 - Longer, hours to days, depending on threat model
 - ID tokens:
 - Short, typically same as access tokens
 - Used immediately at login then discarded
- Both rotatable
 - When an access token expires
 - Refresh produces a new pair
 - New access AND new refresh
- Trade-off
 - Shorter is more secure (smaller window if leaked) but more chatty (more refresh calls)



Scope Minimization

- Request only what you need
 - Scopes describe what the application can do, not what the user can do
 - Each OAuth client should request the smallest set of scopes that lets it function
 - The user grants scopes
 - The application receives a token with only those scopes
 - A token cannot grant more authority than its scopes allow
 - Naming convention
 - Scopes describe capabilities (read:accounts, write:transactions)
 - Not roles (HR_MANAGER)



Roles, Scopes, and Permissions

- Confusing these is the most common OAuth implementation mistake
- Each layer is checked independently
- A request must pass
 - Token has scope
 - User has role
 - Permission allows action

Title: Three Concepts, Three Layers

Concept	Lives at	Describes	Example
Scope	OAuth token	What the application can do	<code>read:accounts</code> , <code>write:transactions</code>
Role	User identity	What the user IS	<code>HR_MANAGER</code> , <code>VIEWER</code> , <code>ADMIN</code>
Permission	Application	What the user can do in this app	"Can transfer up to \$10,000"

Roles, Scopes, and Permissions

- Scopes
 - Are properties of the OAuth token
 - The auth server issues tokens with a fixed set of scopes
 - The ones the user granted
 - Intersected with the ones the client is allowed to request
 - A token's scopes describe what the application can do on behalf of the user
 - Not what the user can do.

Title: Three Concepts, Three Layers

Concept	Lives at	Describes	Example
Scope	OAuth token	What the application can do	<code>read:accounts</code> , <code>write:transactions</code>
Role	User identity	What the user IS	<code>HR_MANAGER</code> , <code>VIEWER</code> , <code>ADMIN</code>
Permission	Application	What the user can do in this app	"Can transfer up to \$10,000"

Roles, Scopes, and Permissions

- Roles
 - Are properties of the user's identity
 - They're typically stored in the application's database or fetched from an identity provider
 - A role describes what the user is in the application's authorization model
 - HR_MANAGER, VIEWER, ADMIN, etc.
 - The same user has the same roles regardless of which application they're using

Title: Three Concepts, Three Layers

Concept	Lives at	Describes	Example
Scope	OAuth token	What the application can do	<code>read:accounts</code> , <code>write:transactions</code>
Role	User identity	What the user IS	<code>HR_MANAGER</code> , <code>VIEWER</code> , <code>ADMIN</code>
Permission	Application	What the user can do in this app	"Can transfer up to \$10,000"

Roles, Scopes, and Permissions

- Permissions
 - Are application-specific authorization decisions
 - They typically combine roles, business rules, and runtime context
 - "the user has the MANAGER role"
 - AND "the requested transfer amount is under \$10,000"
 - AND "it's during business hours, so the action is allowed"
 - Permissions are computed by the application, not granted by the auth server

Title: Three Concepts, Three Layers

Concept	Lives at	Describes	Example
Scope	OAuth token	What the application can do	<code>read:accounts</code> , <code>write:transactions</code>
Role	User identity	What the user IS	<code>HR_MANAGER</code> , <code>VIEWER</code> , <code>ADMIN</code>
Permission	Application	What the user can do in this app	"Can transfer up to \$10,000"

Decision Flow

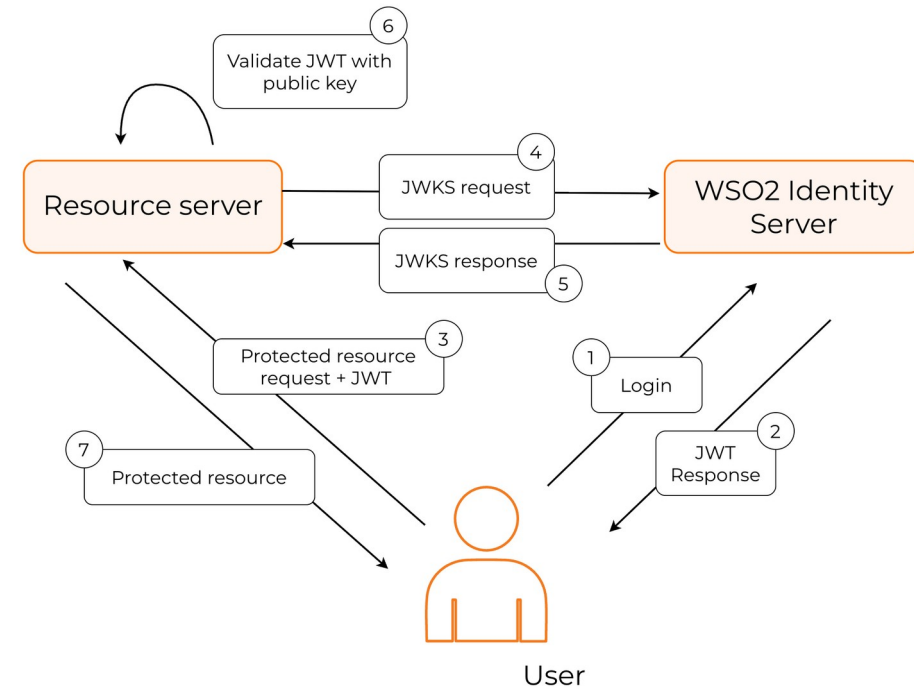
- The decision flow at request time
 - Authentication
 - The token is valid and the user is who they claim to be
 - Scope check
 - The token has the scope required for this API endpoint
 - Role check
 - The user has the role required to even attempt this action
 - Permission check
 - The action is allowed for this specific user given the current context
 - All four must pass for the request to succeed
 - Each layer is a separate check

Title: Three Concepts, Three Layers

Concept	Lives at	Describes	Example
Scope	OAuth token	What the application can do	<code>read:accounts</code> , <code>write:transactions</code>
Role	User identity	What the user IS	<code>HR_MANAGER</code> , <code>VIEWER</code> , <code>ADMIN</code>
Permission	Application	What the user can do in this app	"Can transfer up to \$10,000"

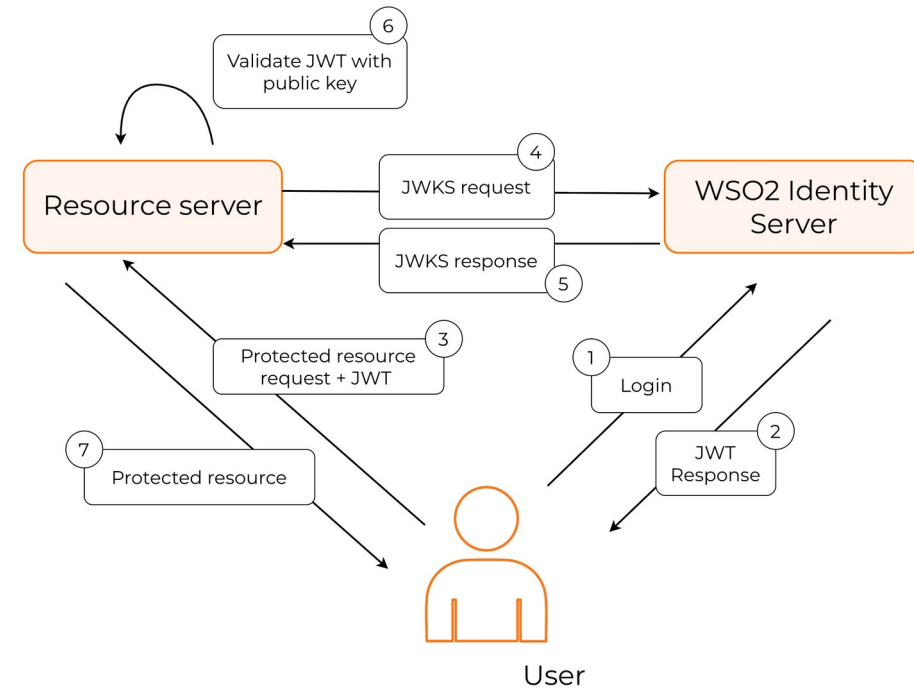
Key Rotation

- JWT verification trustworthiness
 - JWKS = JSON Web Key Set
 - A JSON document listing the public keys the auth server uses to sign tokens
 - Resource servers fetch JWKS at startup and cache it
 - The auth server periodically rotates its signing keys
 - Old keys remain in JWKS during a grace period
 - Spring Security caches JWKS and refreshes it when an unknown kid appears
 - Production-critical
 - Do not write code that hardcodes specific keys



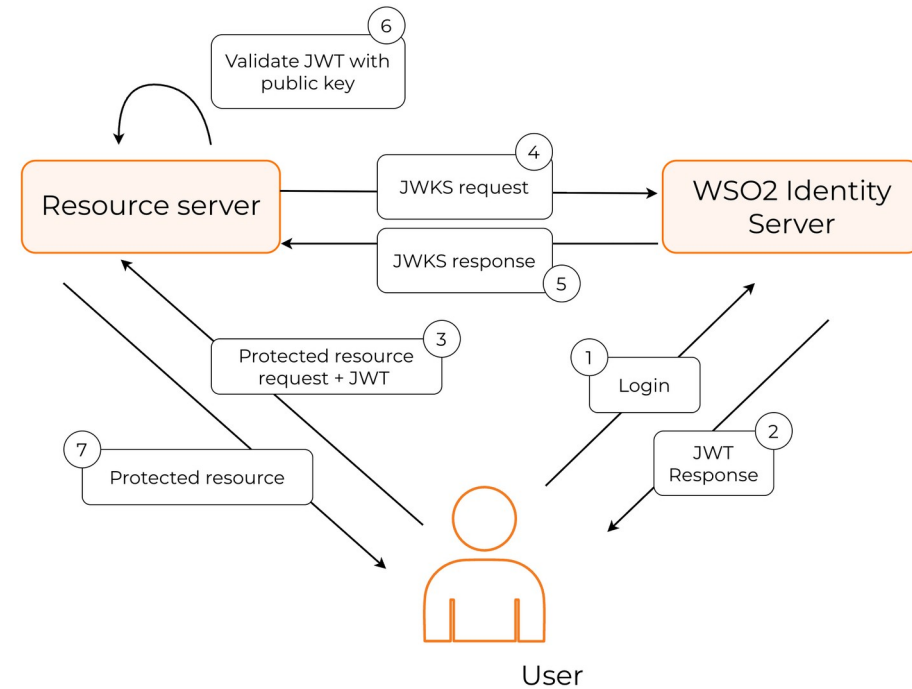
Clock Skew Tolerance

- Server clock mismatch
 - JWTs include iat (issued at), exp (expires at), nbf (not before) all are timestamps
 - Different servers' clocks drift slightly; even with NTP, a few seconds of disagreement is normal
 - Without tolerance, a token issued at 12:00:00 by the auth server might be rejected at 11:59:58 by the resource server
 - Spring Security default
 - 60 seconds of clock skew tolerance; configurable per resource server
 - Production-critical for distributed systems
 - Server saying:
 - "I'll accept tokens up to N seconds outside my view of valid time"



Audience Validation

- The **aud** claim
 - In a JWT identifies which resource server(s) the token is intended for
 - A resource server should reject tokens where **aud** does not include itself
 - Without audience validation
 - A token issued for resource server A could be replayed against resource server B
 - If both trust the same auth server
 - This is a security boundary, not just metadata



Validating Tokens

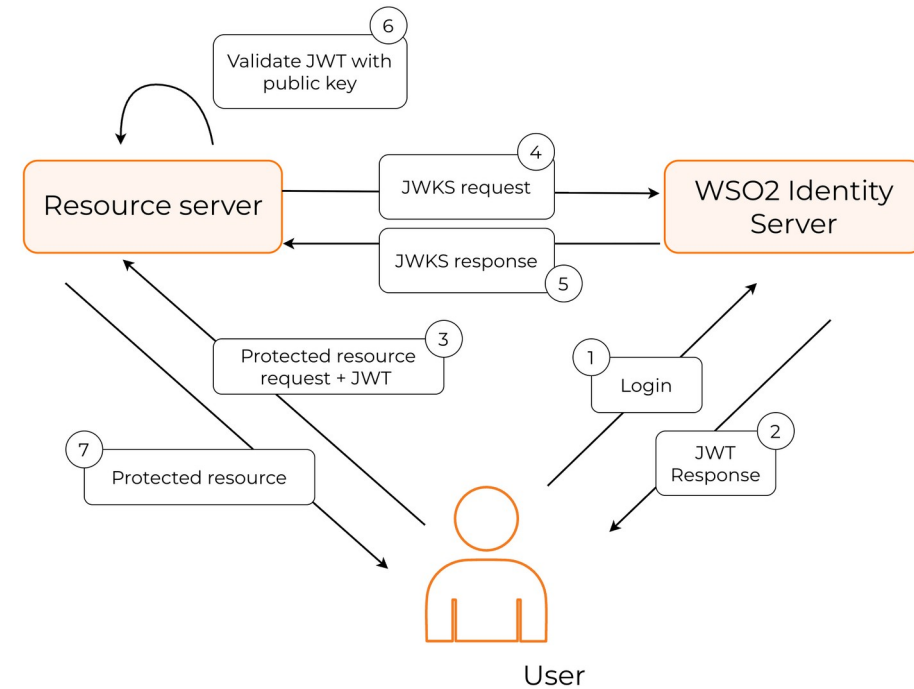
- Token Introspection vs. JWT Verification

- JWT verification

- Resource server validates the signature using cached JWKS
 - No call to auth server per request
 - Fast: pure local operation, no network call
 - Stale: revoked tokens remain valid until they expire

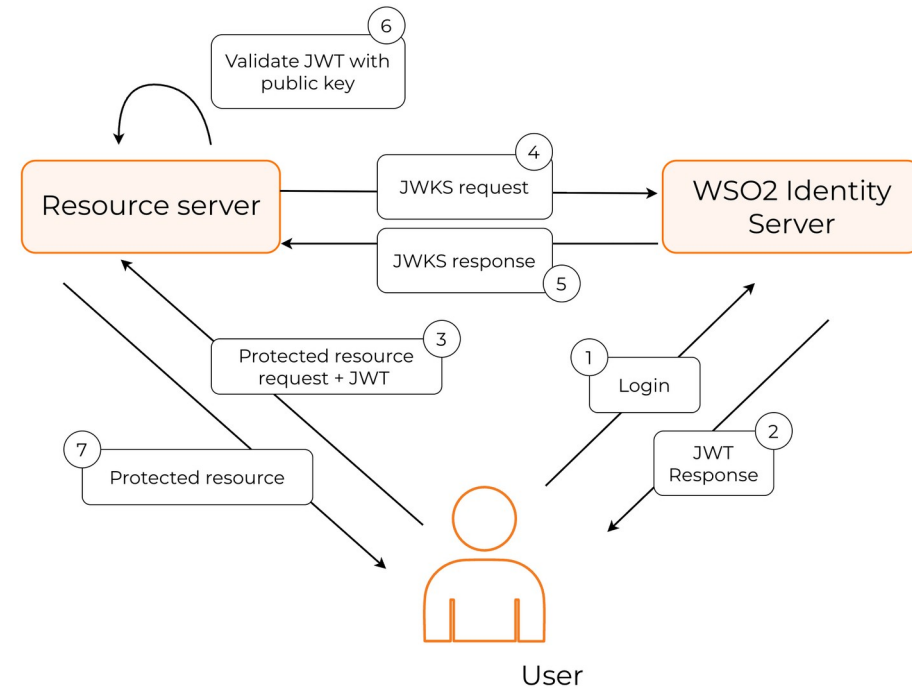
- Token introspection

- Resource server calls the auth server's `/introspect` endpoint for every token
 - Fresh: revocation takes effect immediately
 - Slow: extra network call on every API request
 - The choice depends on threat model
 - How fast must revocation propagate?



Validating Tokens

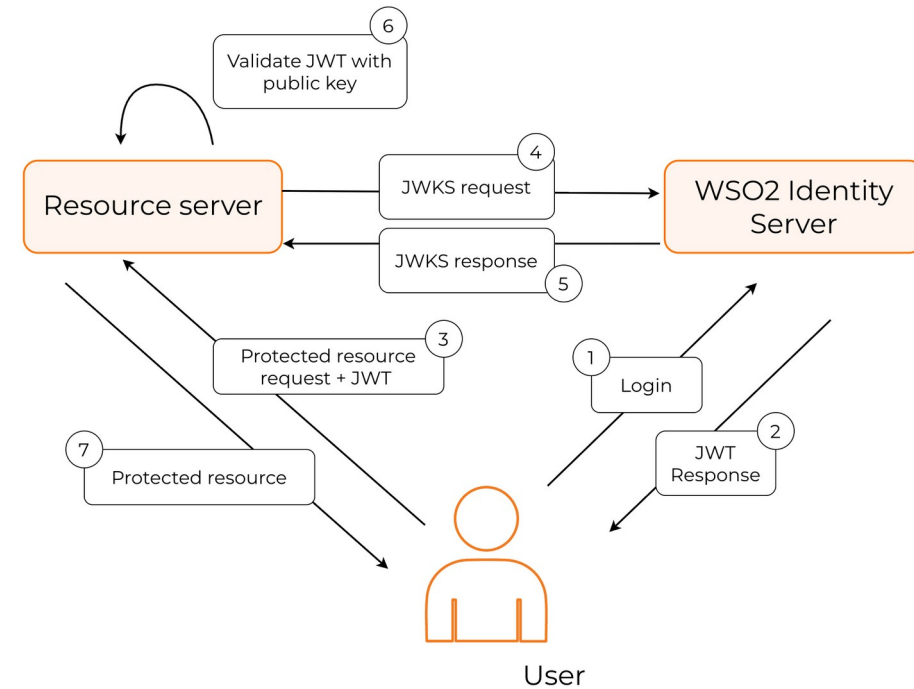
- JWT verification
 - Token is self-contained
 - It includes the user identity, the scopes, the expiry, and a signature
 - The resource server verifies the signature against the cached JWKS and trusts everything inside the token until it expires
 - No call to the auth server per request, this is fast and scales well.
 - The downside
 - Revocation is delayed
 - If a user's account is suspended, their existing tokens remain valid until they expire naturally
 - With longer-lived tokens, the revocation gap can be hours



Validating Tokens

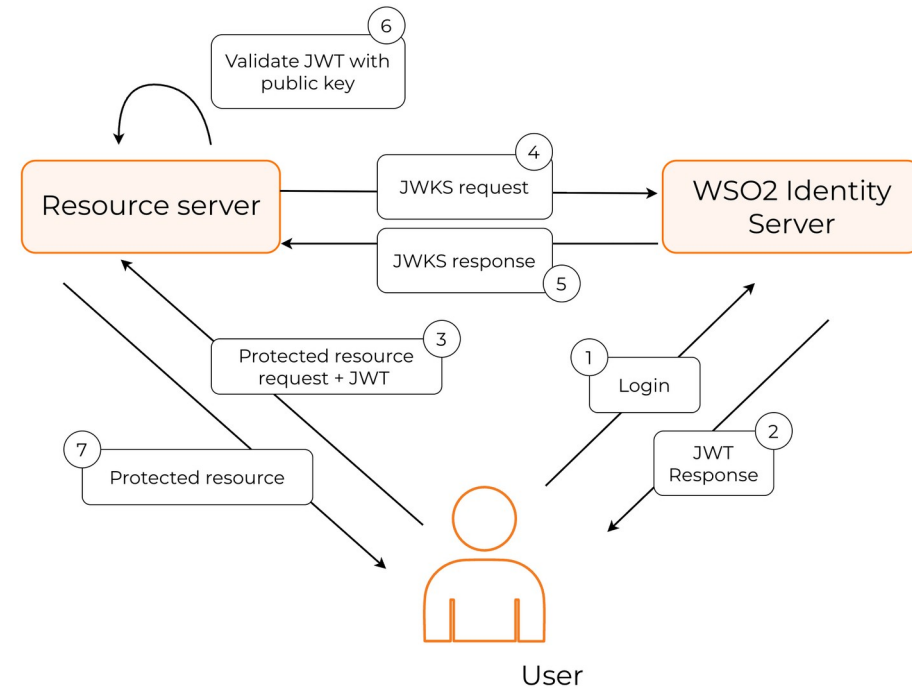
- Token introspection

- The resource server calls the auth server's `/introspect` endpoint for every API request, sending the token
- The auth server checks its current state
 - Is this token still valid?
 - Has the user been suspended?
 - Has the application been disabled?
- Returns either "active" or "inactive"
- The resource server uses the response to decide whether to allow the request.
- The benefit
 - Revocation is immediate
 - Suspend the user, and within milliseconds their next API call fails



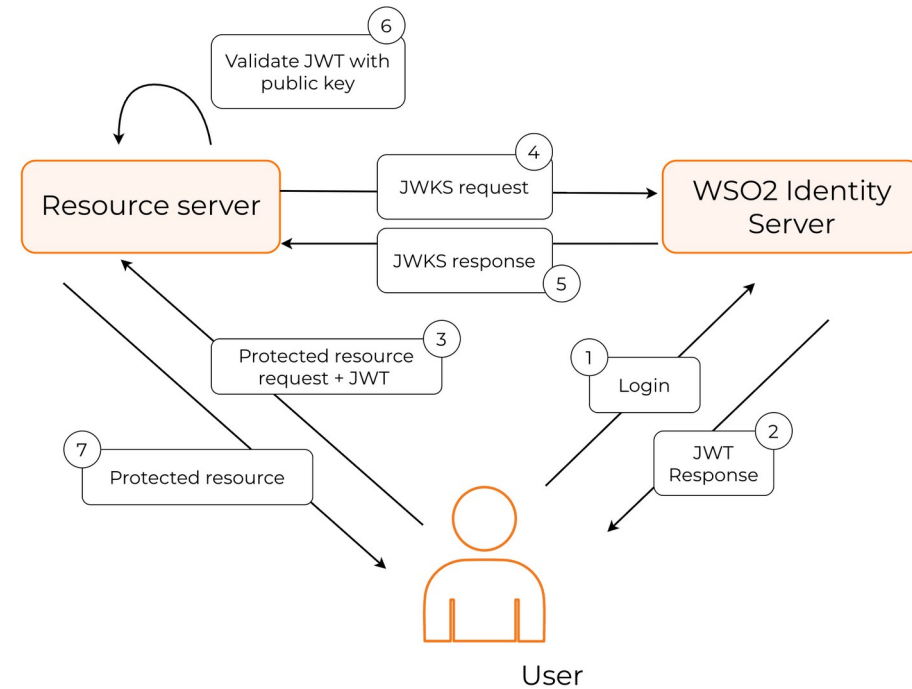
Confused Deputy

- The confused deputy problem
 - A "confused deputy" is a privileged service that performs actions on behalf of less-privileged callers
 - Without scope checks
 - A caller could ask the deputy to do anything the deputy is allowed to do
 - OAuth scopes solve this by attaching the caller's intended capabilities to the request
 - The deputy enforces scope
 - It only does what the token's scopes allow, not what the deputy itself can do



Cookies

- Session management and cookies
 - Session timeout
 - how long an idle session remains valid (15-30 minutes for banking, longer for consumer apps)
 - Sliding vs. absolute expiry
 - Sliding resets on each request, absolute expires regardless of activity
 - Cookie attributes
 - HttpOnly, Secure, SameSite
 - Each closes a different attack vector
 - Concurrent session control
 - How many simultaneous sessions per user
 - Session fixation protection
 - Rotate session ID on login (Spring Security default)



Cookies

- Cookie attributes

- HttpOnly

- JavaScript cannot read the cookie
 - Mitigates XSS

- Secure

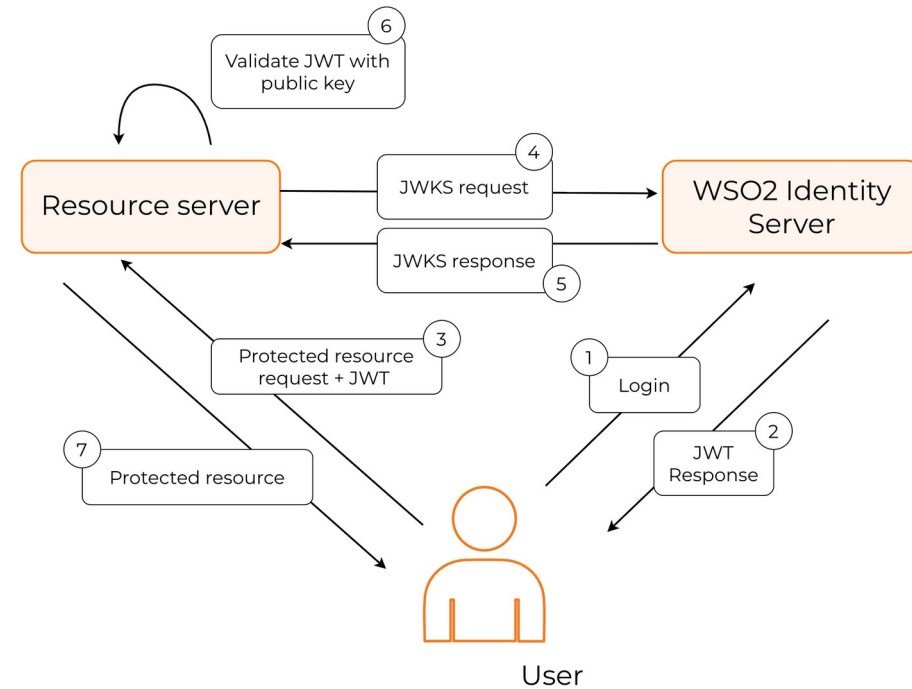
- Cookie only sent over HTTPS
 - Required for production.

- SameSite

- Cookie behavior on cross-site requests.
 - Strict is most secure but can break legitimate flows
 - Lax is the balanced default
 - None requires Secure and is needed for cross-origin authenticated requests.

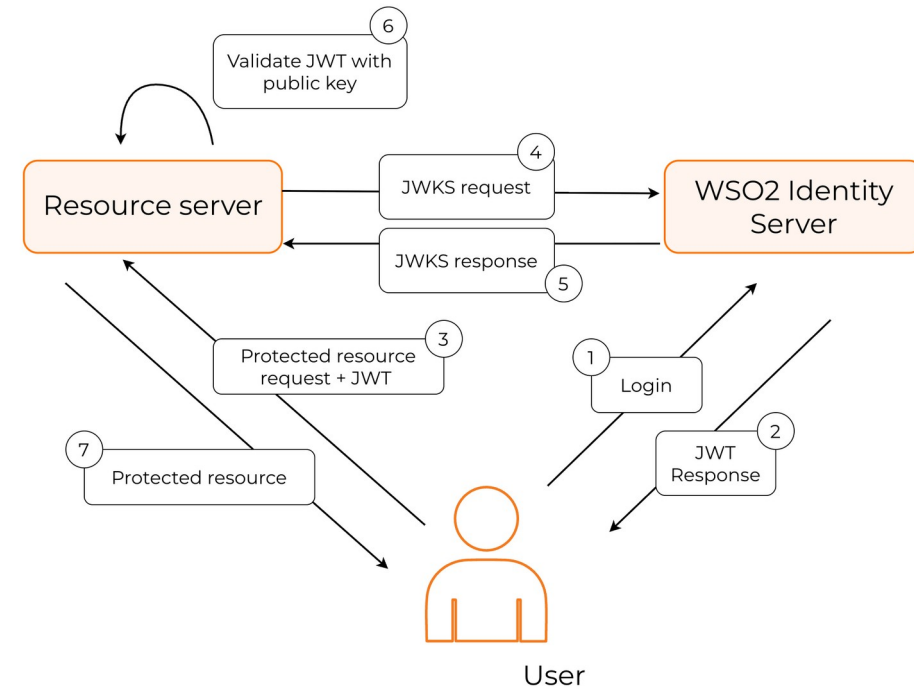
- Domain and Path

- Scope the cookie to the right URLs.



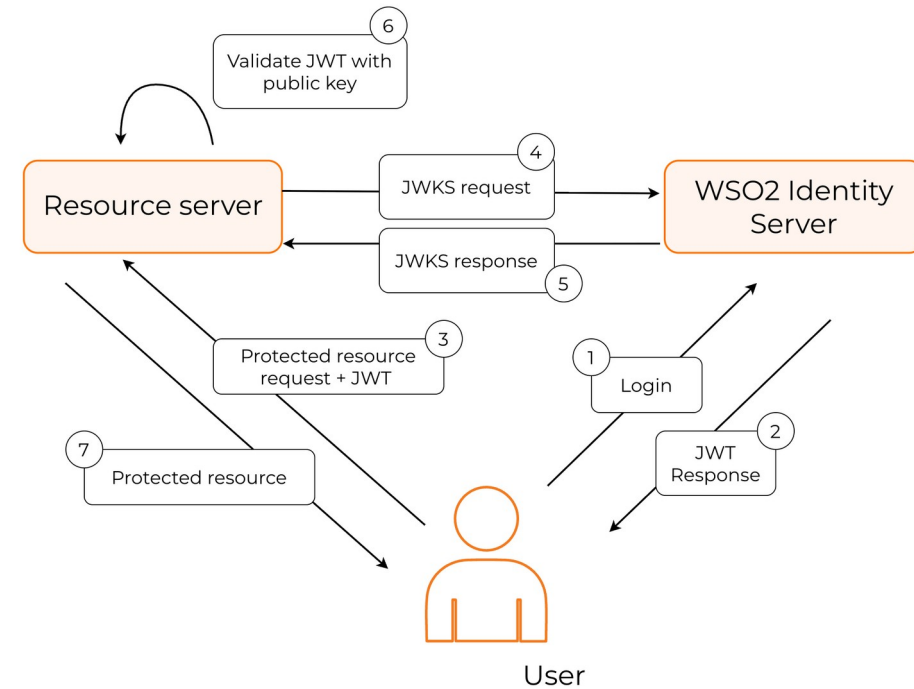
CSRF Revisited

- CookieCsrftokenRepository
 - Stores CSRF token in a cookie readable by JavaScript
 - Sufficient when
 - SPA + BFF + same-origin
 - Trust the JavaScript on the page
 - Not sufficient when
 - XSS bypasses the same-origin assumption
 - User has a malicious browser extension
 - CSRF token is somehow accessible to the attacker



CSRF Revisited

- CookieCsrftokenRepository
 - Compensating controls
 - Content Security Policy
 - Subresource integrity
 - Browser extension policies
- CSRF is one layer of defense, not the only layer



Questions?

